EyeGuard

Scoring & Calculation Explained

This document explains exactly how the EyeGuard system scores CVS risk — covering the CVS-Smart clinical instrument, the rule-based formula, and the Random Forest + Gradient Boosting machine learning pipeline.

Part 1	**CVS-Smart Questionnaire Scoring**	*CVSQ 0–10 · Iqbal et al. (2024)*
Part 2	**Rule-Based Risk Score**	*Symptom score + Environmental modifiers*
Part 3	**Random Forest Classifier**	*Risk level: Low / Moderate / High / Critical*
Part 4	**Gradient Boosting Regressor**	*Risk percentage: 0–100%*
Part 5	**How Both Results Come Together**	*Concordance + final output*

Part 1	CVS-Smart Questionnaire Scoring	*Verbatim instrument · Iqbal et al. (2024) · Score range: 0–10*

The CVS-Smart questionnaire (Iqbal et al., 2024) is a validated five-question clinical screening tool for Computer Vision Syndrome. It is used verbatim — no wording or scoring changes are made. Total score = Q1 + Q2 + Q3 + Q4 + Q5.

Q1 — Visual Complaints

Complaints listed: Blurred vision · Eye strain/fatigue · Double vision · Difficulty refocusing · Near vision discomfort · Unclear objects post-screen · Glare/halos · Diminished vision · Light sensitivity

```
User selects all complaints that apply from the list above.

Count selected = 0 → score 0

Count selected = 1 → score 1

Count selected ≥ 2 → score 2

Max contribution from Q1: 2 points
```

Q2 — Ocular Surface Complaints

Complaints listed: Dry eyes · Eye redness · Irritation/discomfort · Foreign body sensation · Burning/stinging · Itching/rubbing · Watery eye · Heavy eyelids · Frequent blinking

```
User selects all complaints that apply from the list above.

Count selected = 0 → score 0

Count selected = 1 → score 1

Count selected ≥ 2 → score 2

Max contribution from Q2: 2 points
```

Q3 — Extraocular Complaints

Complaints listed: Headache · Neck pain · Shoulder pain · Back pain · Joint pain in fingers/wrists · Inability to hold objects · Difficulty writing · Sleep disturbances · Inattention/depression

```
User selects all complaints that apply from the list above.

Count selected = 0 → score 0

Count selected = 1 → score 1

Count selected ≥ 2 → score 2

Max contribution from Q3: 2 points
```

Q4 — Frequency of Complaints

```
Rare → score 0

Infrequent → score 1

Frequent → score 2
```

Q5 — Screen-Associated Complaints

"Are your complaints associated with the time of screen use?"

```
Never → score 0

Sometimes → score 1

Always → score 2
```

Total CVSQ Score

```
CVSQ Total = Q1 + Q2 + Q3 + Q4 + Q5

Range: 0 to 10 points
```

Score Interpretation (from Iqbal et al., 2024)

CVSQ Score	CVS Status	Recommended Action
7 – 10	Positive CVS Case	Consult ophthalmologist; diagnosis confirmed
5 – 6	High Probability	Confirm diagnosis; follow protective measures
3 – 4	Low Probability	Recheck every 6 months
1 – 2	Not a CVS Case	Follow preventive measures
0	Normal	Preventive measures only

In our study (N = 399): 44.9% scored 7–10 (Positive CVS), 44.1% scored 5–6 (High Probability). Mean CVSQ score = 6.16.

Part 2	Rule-Based Risk Score	*From predict-supabase/route.ts · Fallback when ML is unavailable*

The rule-based engine runs on every submission. It is the fallback when the Flask ML backend is unavailable, and its output is also used as a starting point before the ML models override it. Every formula constant traces to a published source.

Step 1 — Symptom Score (0–100)

Based on Seguí et al. (2015): blurry vision and eye strain are hallmark symptoms (weight 2); headaches and dry eyes are secondary (weight 1).

```
Frequency multiplier:

Never → 0.0

Rarely (1-2 times/week) → 0.5

Sometimes (3-4 times/week) → 1.0

Often (5-6 times/week) → 1.5

Always (every day) → 2.0

cvqRawScore = (eyeStrainFreq × 2) + (blurryVisionFreq × 2)

+ (headachesFreq × 1) + (dryEyesFreq × 1)

Max raw = 2.0 × (2+2+1+1) = 12.0

symptomScore = (cvqRawScore / 12) × 100 [range: 0–100]
```

Step 2 — Environmental Exposure Modifiers

```
Screen Time Modifier (AOA 2016):

< 2h → 0

2h to 6h → ((screenTime - 2) / 4) × 15 [linear 0→+15]

> 6h → 15 + (min(screenTime-6, 6) / 6) × 10 [+15→+25]

Sleep Modifier (NSF 2020 + Sheppard & Wolffsohn 2018):

< 5h → +14 5–6h → +10 6–7h → +6

7–9h → 0 (optimal) > 9h → +2

Brightness Modifier (ISO 9241-303, optimal 40–80%):

< 40% → min(((40 - brightness) / 10) × 1.5, 6) [max +6]

> 80% → min(((brightness - 80) / 10) × 1.5, 6) [max +6]
```

```
Break Modifier (AOA 20-20-20 rule):

Each break taken → -3 pts (capped at -12)
```

Step 3 — Protective Factor Modifiers

```
Blue-Light Filter Usage (Sheppard & Wolffsohn 2018):

Always → -4 Regularly → -2 Sometimes → -1 Never → 0

Exercise Frequency (Scheiman et al. 2011):

Regular → -3 Moderate → -1.5 Minimal → -0.5 None → 0

Outdoor Time (Huang et al. 2015):

> 2h → -3 1–2h → -2 30min–1h → -1 < 30min → 0

totalModifier = screenMod + sleepMod + brightnessMod + breakMod

+ blueLightMod + exerciseMod + outdoorMod
```

Step 4 — Final Risk Score

```
IF symptomScore = 0:

riskScore = clamp(totalModifier, 0, 35)

[No symptoms → exposure-only warning, capped at 35%]

ELSE:

riskScore = clamp(symptomScore + totalModifier, 0, 100)

[Symptoms drive the score; environment scales it up or down]
```

Step 5 — Risk Level Thresholds

Risk Score	Risk Level	Meaning
0 – 24%	Low	Minimal risk; maintain current habits
25 – 49%	Moderate	Habits need attention; early warning
50 – 74%	High	Strong risk pattern; symptoms likely present
75 – 100%	Critical	Severe risk; urgent habit change needed

Step 6 — Fatigue Score (0–10, independent)

```
symptomFatigue = (cvqRawScore / 12) × 5 [0–5 pts]

screenFatigue = min(screenTime / 3, 2.5) [0–2.5 pts, max at 7.5h]
```

```
sleepFatigue = sleepHours < 6 ? 1.5 : sleepHours < 7 ? 0.8 : 0

breakFatigue = 0 breaks → 1.0 pt | 1 break → 0.5 pt | 2+ → 0

fatigueScore = clamp(symptomFatigue + screenFatigue
+ sleepFatigue + breakFatigue, 0, 10)
```

Part 3	Random Forest Classifier	*Predicts risk level: Low / Moderate / High / Critical*

When the Flask ML backend is running, the Random Forest classifier overrides the rule-based risk level. It takes 12 engineered features from the behavioral survey and votes across 200 decision trees.

Model Configuration

Algorithm	Random Forest Classifier (scikit-learn RandomForestClassifier)
Number of trees	200 estimators
Max tree depth	15
Min samples split	5
Min samples leaf	2
Scaling	StandardScaler applied before fitting
CV performance	~85% accuracy (5-fold cross-validation)

The 12 Input Features

#	Feature Name	Derived From	Range
1	screen_time_minutes	Academic + non-academic screen time (converted to mins)	0–720
2	break_minutes	Break frequency × session duration	0–...
3	symptom_index	Weighted avg severity of reported symptoms	0–1
4	sleep_quality	Sleep hours / 10	0–1
5	water_intake_ratio	Cups of water / 8 (neutral default = 6 cups)	0–1
6	break_effectiveness	Break type (outdoor=0.9, rest=0.6) × duration factor	0–1
7	break_ratio	Break time vs recommended (5min/hr)	0–1
8	symptom_count	Number of symptoms reported above Never	0–10
9	had_outdoor_break	Whether an outdoor break was taken (binary)	0/1
10	eye_exercises_count	Number of eye exercises done	0–4
11	blue_light_filter	Blue light filter on/off (binary)	0/1
12	screen_time_ratio	Screen time as % of waking day	0–1

How Random Forest Classifies Risk Level

```
1. Features are standardized using StandardScaler (mean=0, std=1)
```

```
2. 200 decision trees are trained on bootstrap samples of training data
3. Each tree independently predicts one of: Low(0), Moderate(1), High(2), Critical(3)
4. Final prediction = majority vote across all 200 trees
5. Confidence = probability assigned to the winning class (via predict_proba)

predict_proba output example:
{ low: 0.05, moderate: 0.12, high: 0.61, critical: 0.22 }
→ Predicted class: High | Confidence: 0.61
```

Symptom Severity Weights (for symptom_index feature)

Symptom	Severity Weight	Rationale
Severe eye strain	0.8	Hallmark CVS symptom — highest weight
Blurred vision	0.7	Hallmark CVS symptom
Eye fatigue	0.7	Direct visual fatigue indicator
Difficulty focusing	0.65	Accommodative demand indicator
Headache	0.6	Secondary CVS symptom
Dry eyes	0.55	Ocular surface symptom
Moderate eye strain	0.5	Mid-severity strain
Mild eye discomfort	0.3	Low-severity indicator

`symptom_index = min(1.0, mean(severity_weights of reported symptoms))`

Part 4	Gradient Boosting Regressor	*Predicts exact risk percentage: 0–100%*

The Gradient Boosting regressor uses the same 12 engineered features as the Random Forest but outputs a continuous value instead of a class. It builds trees sequentially — each tree corrects the errors of the previous one — making it well-suited for the non-linear relationship between screen exposure and risk (risk escalates sharply beyond 6 hours daily).

Model Configuration

Algorithm	Gradient Boosting Regressor (scikit-learn GradientBoostingRegressor)
Estimators	200 sequential trees
Learning rate	0.1 (step size for each tree's contribution)
Max tree depth	8
Subsample	0.8 (each tree uses 80% of data — reduces overfitting)
Loss function	Squared error (minimises mean squared error)
Scaling	StandardScaler applied before fitting
CV performance	$R^2 \approx 0.82$ · RMSE ≈ 6.4% (5-fold cross-validation)

How Gradient Boosting Computes Risk Percentage

```
1. Features are standardized using StandardScaler
2. Model starts with a constant prediction (mean of training labels)
3. For each of 200 iterations:
a. Compute residual errors from current prediction
b. Fit a shallow decision tree to predict those residuals
c. Update prediction: new = old + (learning_rate × tree_prediction)
4. Final prediction = sum of all tree contributions + initial constant
5. Output is clamped to valid range: clamp(prediction, 0, 100)

Example gradient boosting update:
iteration_1: prediction = 45.2
iteration_2: residual = actual - 45.2 → tree learns +3.1
new prediction = 45.2 + (0.1 × 3.1) = 45.51 ...(continues 200x)
```

Why Gradient Boosting for Percentage, not Classification?

Property	Random Forest (classifier)	Gradient Boosting (regressor)
Output type	Discrete class (Low/Moderate/High/Critical)	Continuous number (e.g. 63.4%)
Best for	When clear categories are needed	When a precise score matters
Risk non-linearity	Handles well via tree splits	Handles better — sequential correction

Property	Random Forest (classifier)	Gradient Boosting (regressor)
Interpretability	Feature importance, probability per class	Feature importance, partial dependence
CV metric	Accuracy (~85%)	R^2 (~0.82)

Break Effectiveness Calculation (used in both models)

```
Break type effectiveness weights:

Outdoor = 0.9 Walk = 0.85 Exercise = 0.8 Meditation = 0.75

Rest = 0.6 Social media = 0.3 Gaming = 0.2

Duration multiplier:

< 5 min → duration / 5

5–15 min → 1.0 (optimal)

15–30 min → 0.9

> 30 min → 0.7

break_effectiveness = min(1.0, type_weight × duration_multiplier)
```

Part 5	How Both Results Come Together	*CVSQ clinical score + ML prediction → concordance check → final output*

Complete Prediction Flow

Stage 1	**CVSQ**	User answers 5 CVS-Smart questions → CVSQ total (0–10) + CVS status tier
Stage 2	**Rule-Based**	Behavioral survey answers → symptom score + modifiers → rule-based risk score (0–100%) + risk level
Stage 3	**ML Override**	IF Flask backend available: Random Forest → replaces risk level Gradient Boosting → replaces risk percentage ELSE: rule-based values are kept
Stage 4	**Concordance**	CVSQ tier vs ML risk tier — do they agree? → concordant = true/false + explanation note
Stage 5	**Output**	Results page shows: Card A: CVSQ score + CVS status Card B: Risk % + Risk level + Fatigue score Banner: Concordance status + recommendations

Concordance Rules

A prediction is concordant when the ML risk tier matches what the CVSQ clinical tier expects. Because CVSQ is an externally validated instrument, concordance rate is the system's primary defensible accuracy metric.

CVSQ Score	CVSQ Status	Expected ML Risk Tier	Concordant If
7–10	Positive CVS	High or Critical	ML → High or Critical
5–6	High Probability	Moderate, High, or Critical	ML → Moderate / High / Critical
3–4	Low Probability	Low or Moderate	ML → Low or Moderate
0–2	Not CVS / Normal	Low	ML → Low

When the Two Results Disagree (Discordance)

Scenario A	**High CVSQ, Low ML Score**	User has active symptoms per CVSQ but reports relatively healthy habits. System defers to CVSQ — raises risk level to at least High. User is told: symptoms match a CVS pattern even if habits appear manageable.
Scenario B	**Low CVSQ, High ML Score**	No significant symptoms yet, but behavioral habits are high-risk. System keeps ML warning — labels it 'pre-symptomatic risk' capped at Moderate. User is told: this is an early warning before symptoms appear.

Accuracy Metric

```
Concordance Rate (%) = (Concordant Predictions / Total Predictions) × 100

In our study (N = 399):
```

```
CVSQ mean score: 6.16 / 10

89.0% of respondents scored ≥ 5 (High Probability or Positive CVS)

This validates the ML model's risk output against a published clinical instrument
```

EyeGuard Scoring Reference · CVSQ: Iqbal et al. (2024) Med Hypothesis Discov Innov Ophthalmol · Symptom weights: Seguí et al. (2015) J Clin Epidemiol · Screen time thresholds: AOA (2016) · Sleep norms: NSF (2020) · Brightness: ISO 9241-303 · ML: scikit-learn RandomForestClassifier + GradientBoostingRegressor